Link state Routing Algorithm for obtaining routing tables at each node

Aim: To write a C program to implement Link state Routing Algorithm for obtaining routing tables at each node.

Description:

The **Link State Routing (LSR) algorithm** is a dynamic routing protocol used in computer networks, where each router independently calculates the shortest path to every other router in the network. Unlike the Distance Vector Routing algorithm, which only communicates with neighbouring routers, Link State Routing involves each router having a complete view of the network topology to make routing decisions.

How Link State Routing Works Step-by-Step

1. **Initial Setup**: Each router detects its directly connected neighbors and calculates the cost of each link.
2. **LSA Creation and Flooding**: Each router creates an LSA with information about its directly connected neighbors and their link costs. It then floods this LSA to all routers in the network.
3. **Building the Link State Database**: Each router receives LSAs from every other router and builds an identical LSDB that represents the full network topology.
4. **Running Dijkstra's Algorithm**: Each router applies Dijkstra's algorithm to its LSDB to find the shortest path to each other router. This process generates the router's **routing table**, which provides the optimal path for each destination.
5. **Updating Routes on Changes**: When a link goes up or down, the affected router immediately sends an updated LSA, and the network recalculates paths accordingly.

Protocols Using Link State Routing : The most commonly used protocol that applies Link State Routing is **OSPF (Open Shortest Path First)**, widely used in enterprise networks. **IS-IS (Intermediate System to Intermediate System)** is another link-state protocol used primarily in large service provider networks.

Program:

```
#include <stdio.h>
#define MAX_NODES 10
#define INF 99

void dijkstra(int graph[MAX_NODES][MAX_NODES], int src, int n)
{
   int dist[MAX_NODES];
   int visited[MAX_NODES] = {0};

   // Initialize distances
   for (int i = 0; i < n; i++)
  {
      dist[i] = INF;
   }
   dist[src] = 0;

   for (int count = 0; count < n - 1; count++)
  {
      int minDist = INF, u = -1;

      // Find the unvisited node with the smallest distance
```

```
        for (int i = 0; i < n; i++)
       {
            if (!visited[i] && dist[i] < minDist)
          {
                minDist = dist[i];
                u = i;
            }
        }

        visited[u] = 1;

        // Update distances for neighbors of the chosen node
        for (int v = 0; v < n; v++)
       {
            if (!visited[v] && graph[u][v] && dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v])
           {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    // Print the results
    printf("Node \t Distance from Source %d\n", src);
    for (int i = 0; i < n; i++)
  {
        printf("%d \t\t %d\n", i, dist[i]);
    }
}

int main()
{
    int n;
    int graph[MAX_NODES][MAX_NODES];

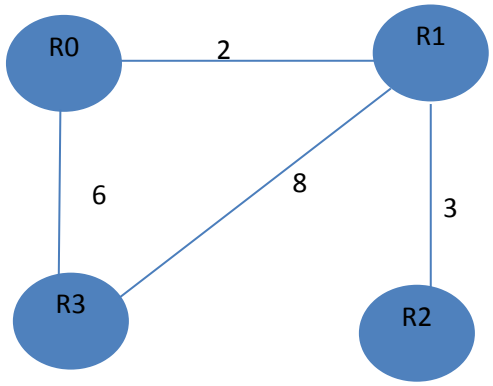
    printf("Enter the number of nodes in the network: ");
    scanf("%d", &n);

    printf("Enter the adjacency matrix (use 0 for no direct link):\n");
    for (int i = 0; i < n; i++)
 {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
            if (i != j && graph[i][j] == 0)
            {
                graph[i][j] = INF;
            }
        }
    }

    int src;
    printf("Enter the source node: ");
    scanf("%d", &src);
```

```
    dijkstra(graph, src, n);

    return 0;
}
```



Input to the Program:
Enter the number of nodes in the network: 4
Enter the adjacency matrix (use 0 for no direct link):
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0

Output of the Program:

Enter the source node: 0

Node	Distance from Source 0
0	0
1	2
2	5
3	6

Input to the Program:
Enter the number of nodes in the network: 4
Enter the adjacency matrix (use 0 for no direct link):
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
0 2 0 6

Enter the source node: 1

Node	Distance from Source 1
0	2
1	0
2	3
3	8